MusicMood API

Technical Report

Leah-Megan Chi

201698979

GitHub: https://github.com/LeahChi/music_mood_api

Live API Docs: https://musicmoodapi-production.up.railway.app/docs

Project Overview

‘MusicMood’ is a music listening API that combines a user’s listening history with real Spotify audio features to generate mood listening analytics and AI-powered reflective summaries. Users can log listening sessions against tracks from a 5000-track database and can receive non-clinical mood interpretations generated by Llama 3 via the Groq API.

This project idea was motivated by a genuine interest in the intersection of music psychology and software engineering. MusicMood explores how quantitative audio features (valence, energy and danceability) can be used to generate personalised insights grounded in psychological frameworks. The system is built with FastAPI, SQLite, SQL Alchemy and JWT authentication. It’s deployed on Railway and includes a fully functional UI server directly from the API.

Architecture and Design Choices

This system follows a layered RESTful architecture with five distinct concerns: authentication, data management, session logging, analytics computation and AI interpretation. This separation ensures each layer can be understood, tested and extended independently. A Diagram illustrating the architecture can be found in Appendix A.

The database is comprised of four related user models: User, Track, ListeningSession and MoodSnapshot. The Track model stores real Spotify audio features sourced from a Kaggle dataset of 5000 tracks. The ListeningContext enumeration (working, commuting, relaxing, exercising, socialising and sleeping) was grounded in S Schäfer et al’s [1] research on the psychological functions of music listening in everyday life, ensuring the data model had academic justification rather than just arbitrary design choices.

The most significant architectural decision was the design of the AI endpoints. Rather than passing raw data into the language model, the system first computes aggregated analytics, and passes these structured summaries to Llama 3 for interpretation. This ensures the AI acts as a presentation layer over the system’s own computation rather than a simple LLM wrapper, which is more technically defensible and a unique take on how we can use GenAI creatively.

Mood scoring is grounded in Russell’s (1980) circumplex model of affect [2], which maps emotional stated onto two dimensions: valence (positivity) and arousal (energy). This produces four quadrants: Excited (high valence, high energy), Calm (high valence, low energy), Tense (low valence, high energy), and Melancholic (low valence, low energy). These quadrants were used consistently across all analytics and AI endpoints.

Technology Stack Justification

FastAPI was chosen over Django REST Framework for several reasons. It is a modern, async-first Python framework that auto-generates OpenAPI (Swagger) documentation from type annotations, providing accurate, interactive API documentation. Its use of Pydantic for request and response validation enforces data integrity at the schema level. Compared to Django, FastAPI is more lightweight and better suited to API-only projects that do not require a full MVC structure.

SQLite was chosen because it requires no server setup and creates a local database file automatically on startup. SQLAlchemy provides an ORM layer that abstracts raw SQL into Python objects, making queries readable and maintainable.

JWT authentication was implemented using HS256 (HMAC-SHA256), defined in RFC 7519. HS256 is appropriate for a single-server application where both token generation and verification are handled by the same service. RS256 would be more suitable in a distributed system where multiple independent services need to verify tokens independently, this represents a future improvement as the system scales. Passwords are hashed using bcrypt, which is intentionally slow to resist brute-force attacks.

The Groq API was chosen to access Llama 3.3 70B as a free, reliable alternative to paid models.

Testing and Error Handling

All endpoints were tested manually through Swagger UI with both valid and invalid inputs. Test cases included: successful registration and login, duplicate username rejection (400), invalid email format (422), wrong password rejection (401), accessing another user's data (403), invalid listening context (400), and out-of-range audio feature values (422).

Global error handlers were implemented in main.py to ensure all endpoints return clean, consistent JSON error responses rather than FastAPI's default output. Pydantic validators enforce constraints at the schema level e.g. valence, energy, and danceability must be between 0.0 and 1.0. The AI endpoints include fallback logic that returns a rule-based summary if the Groq API is unavailable, ensuring the endpoint always returns a valid response regardless of external API status.

Challenges and Lessons Learnt

The most significant technical challenge was designing the AI endpoint in a way that was academically defensible. Early iterations simply forwarded user data to the language model, which risked the endpoint appearing as an arbitrary LLM wrapper. The solution was to restructure the architecture so that the system first computes its own only then passes these structured summaries to Llama 3 for interpretation. This required careful thought about the separation of concerns between the analytics layer and the AI layer.

Deployment to Railway presented ASGI compatibility challenges, as Railway's default build system required explicit Procfile configuration to support FastAPI's async architecture.

A key lesson learned was the importance of early deployment, attempting to deploy late in the project revealed environment differences that were time-consuming to resolve. I believe future projects would benefit from deploying a minimal version on day one. The GenAI-assisted development workflow also produced an important insight: using AI to deeply understand concepts, such as Russell's circumplex model and JWT algorithm trade-offs, rather than simply generating code, produced more academically grounded and defensible design decisions.

Limitations and Future Improvements

The seed data refers to pre-populated demonstration data loaded into the database at startup. In a production environment, all sessions would be generated exclusively through real user interaction with the POST /sessions endpoint. This means the analytics for demonstration users do not reflect genuine listening behaviour, which is a noted limitation.

The AI interpretation is non-clinical and based on patterns in audio features rather than verified psychological data. This is clearly disclaimed in every AI response and in the API documentation. The system makes no diagnostic claims and is intended purely as a reflective tool.

Future improvements would include Spotify OAuth2 integration to automatically pull real listening history from users' accounts using the /me/player/recently-played endpoint, replacing manual session logging entirely. A lightweight Spotify track lookup endpoint could automatically populate audio features from a Spotify ID, removing the need for manual entry. Finally, a more sophisticated mood model could incorporate time-of-day patterns and sequential listening behaviour rather than simple averages.

Generative AI Declaration

This project used Claude (Anthropic) as the primary GenAI tool. Claude was selected for its strong architectural reasoning capabilities.

The most valuable use of Claude was not code generation but conceptual understanding — deeply engaging with Russell's circumplex model, JWT algorithm tradeoffs, and the architectural distinction between an LLM wrapper and a genuine AI interpretation layer. This understanding meant every design decision could be academically justified rather than simply copied.

The most significant creative application was the AI endpoint architecture. It’s deliberately designed so that Llama 3 interprets pre-computed analytics from the system's own database, rather than acting as a general-purpose chatbot. Selected conversation log screenshots are included as evidence of GenAI usage – In Appendix A.

References

[1] Schäfer, T., Sedlmeier, P., Städtler, C. and Huron, D. (2013) 'The psychological functions of music listening', Frontiers in Psychology, 4, p. 511. Available at: https://pmc.ncbi.nlm.nih.gov/articles/PMC3741536

[2] Russell, J.A. (1980) 'A circumplex model of affect', Journal of Personality and Social Psychology, 39(6), pp. 1161–1178. Available at: https://www.researchgate.net/figure/Russells-circumplex-model-of-affect

[3] Spotify (2024) Audio Features — Web API Reference. Available at: https://developer.spotify.com/documentation/web-api/reference/get-audio-features

[4] Pandya, M. (2023) Spotify Tracks Dataset. Kaggle. Available at: https://www.kaggle.com/datasets/maharshipandya/-spotify-tracks-dataset

[5] Internet Engineering Task Force (2015) RFC 7519: JSON Web Token (JWT). Available at: https://datatracker.ietf.org/doc/html/rfc7519

[6] Mozilla Developer Network (2024) HTTP Response Status Codes. Available at: https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Status

Appendix A

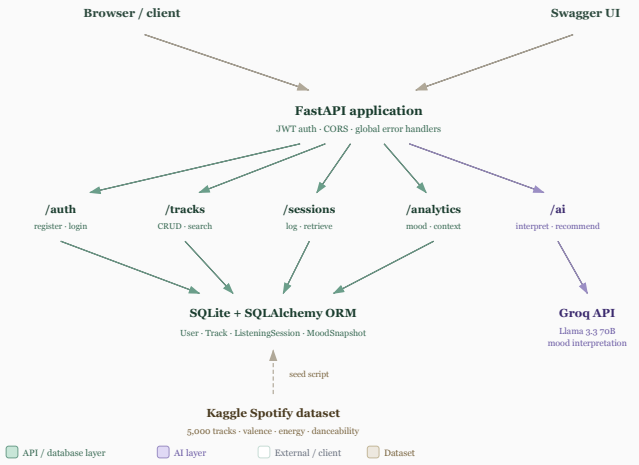

Figure 1- Architecture

lets say i did my musicmood idea and i only got 75, what would you say went wrong, lets look at risks and mitigate them

Really smart to do this now. Let me think like an examiner who just gave you 75 and wrote the feedback email.

Why you got 75 and not 90+

🔴 **Risk 1: The AI endpoints felt bolted on**

What went wrong: `/ai/interpret` just wraps Claude with a basic prompt. The examiner asks *"how does the AI actually use your database?"* and the answer is *"it just summarises the JSON I pass it."*

Mitigation: The AI endpoint must visibly query your DB, compute intermediate analytics, then pass *structured, meaningful data* to Claude — not raw JSON dumps. The prompt engineering itself should be sophisticated and documented. Show the examiner the prompt in your report.

Figure 2 - GenAI conversation log

im thinking of implementing AI into my project to provide a summary of the user's listening 'habits'. whats the best way to go about this

Great idea — this could be a really strong differentiator for your project. Here's how I'd approach it:

The key design decision is that the AI should never see raw data. Instead your API should first compute aggregated analytics, then pass those structured summaries to the model. This means:

```
Request → Query DB → Compute aggregates → Build structured prompt → LLM → Return JSON
```

This makes it academically defensible — the AI acts as an **interpretation layer** over your own system's computation, not just a chatbot wrapper.

What to pass to the model:

- Average valence, energy, danceability over last N sessions
- Most common listening context (working, commuting etc.)
- Top genres and their mood scores
- Mood trend direction (improving, declining, stable)

Recommended stack:

- **Groq API** — free, fast, gives you Llama 3.3 70B
- Returns responses in under a second which is great for a live demo

Figure 3 - GenAI conversation log